Abstract Data Types

An **abstract data type (ADT)** or **interface** *defines* the operations that a data structure supports. These define the algorithms that *must* be implemented for the data structure to work. Basically this describes *what* a data structure is able to do. They are a sort of *standards* specification that must be implemented to be used and to be considered as a specific data structure.

Implemented abstract data types are called **implementations**. Implementations describe *how* a data structure does what it does. These define *how* the data is organized internally and the definitions of the algorithms described in the abstract data type. There can be *many* implementations of the same abstract data type which offer different performance benefits that we can compare using asymptotic notation. Furthermore, we'll see that some of these abstract data types can be *combined* into a single implementation.

Below we'll list the abstract data types that we'll tackle in this course.

Queue, Stack, Deque ADTs

These 3 ADTs are grouped together due to *how* similar these are to each other. In a way these ADTs actually build upon each other.

The **Queue** ADT represents a collection of elements to which we can add more elements and remove the next element. The algorithms that must be implemented for a Queue are:

- `add(x)` : add the value `x` to the Queue
- `remove()` : remove the next (previously added) value from the Queue and return it

> Note that some texts name the operations differently. For instance, some might call the `remove()` operation `pop()`, and etc.

Implementing just these 2 operations are enough to implement the Queue ADT. Notice that the `remove()` operation takes no argument. As to which element is removed from the Queue is determined by the Queue's **queueing discipline**. There are multiple possible queueing disciplines, the common of which are FIFO, priority, and LIFO.

The **first-in-first-out** (FIFO) queueing discipline acts just like a regular queue at the grocery or enrolment. Elements are removed in the same

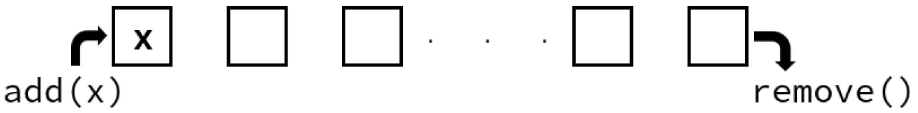


Figure 2: FIFO Queue

order as they were added in (fig. 2). This is the *most* common type of Queue and hence the qualifier FIFO is often omitted. When we say Queue, we mean to say a FIFO Queue.

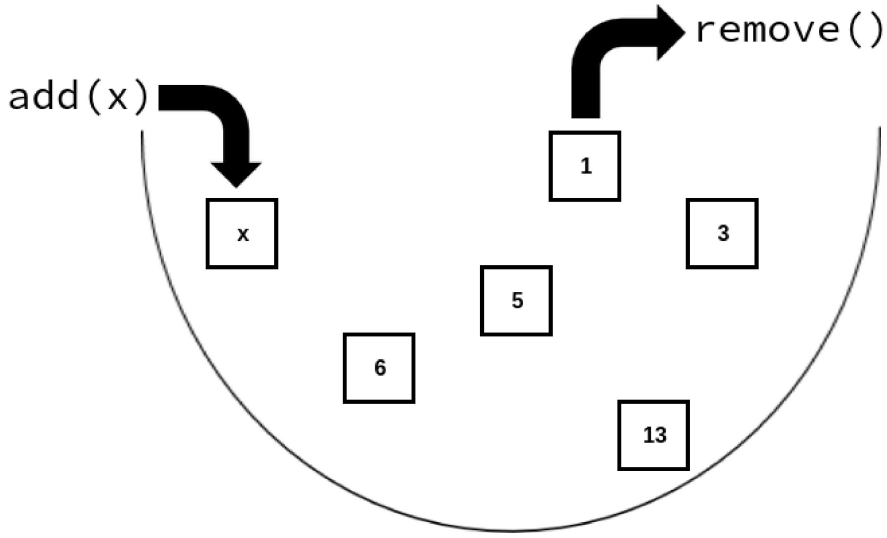


Figure 3: Priority Queue

A **priority** queueing discipline always removes elements based on a *priority*. In most cases, removing the smallest value from the queue (fig. 3). Unlike the FIFO queueing discipline, the order in which elements are added do not determine the next removed value. Rather, the *value* of each element determines which element gets removed next. This is akin to how patients are triaged in the emergency room. Patients arrive at the emergency room one-by-one, are evaluated one-by-one, and when a doctor becomes available, will treat the patient with the most life-threatening condition first.

A **last-in-first-out** (LIFO) queueing discipline is similar to the FIFO queue, in that the order of the added elements determine the next removed element. In the case of the LIFO queueing discipline, the most recently added element is removed next. This is best represented by a *stack* of plates (fig. 4). In fact, this type of queueing discipline is so common it gets its own name—a **Stack**. In a Stack ADT, the operations `add(x)` and `remove()` are also often replaced by `push(x)` and `pop()` to reduce confusion between the Queue and Stack.

A **Deque** is a generalization of both Queue and Stack, it is a sort of *"looser"* specification of both. A Deque represents a *sequence* of ele-

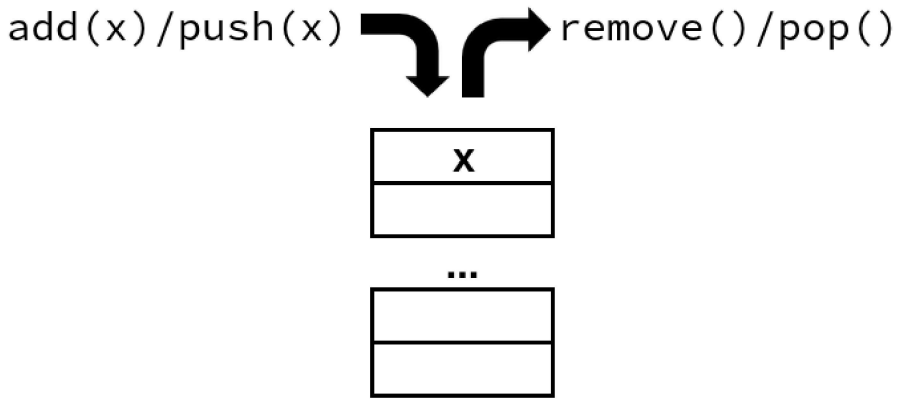


Figure 4: LIFO Queue

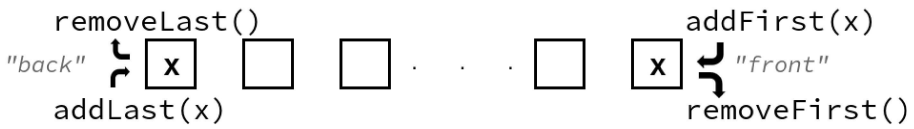


Figure 5: Deque

ments that have a "front" and a "back" (fig. 5); the elements can be added/removed to/from either ends. The operations of a Deque are self-explanatory: `addFirst(x)`, `removeFirst()`, `addLast(x)`, `removeLast()`. Notice that a Stack can be implemented with a Deque by utilizing only `addFirst(x)` and `removeFirst()`, while a FIFO Queue can be implemented by utilizing only `addLast(x)` and `removeFirst()`.

Lists

The **List** ADT represents a *sequence,* $x_0, x_1, x_2, ..., x_{n-1}$, that is quite similar to the Deque ADT. Lists are a *further* generalization of the Deque ADT. They are almost the same, but the List offers more robust operations—**indexing**. Indexing allows us to access any arbitrary element in the sequence, just like in an array. The operations available on a List are as follows:

- `size()`: return `n`, the length of the list
- `get(i)`: return value of x_i in the sequence
- `set(i, x)`: set the value of x_i in the sequence to x
- `add(i, x)`: insert the value x at the position x_i, displacing the positions of $x_i, ..., x_{n-1}$ by $+1$, and increment `n`.
- `remove(i)`: remove and return the value x_i from the sequence, displacing the positions of $x_{i+1}, ..., x_{n-1}$ by -1, and decrement `n`.

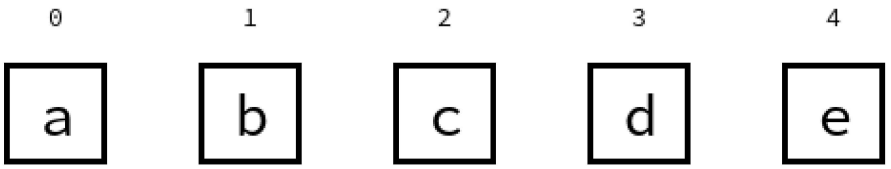


Figure 6: Initial list example

To make the description of `add(i, x)` and `remove(i)` clearer, let's have an example. Consider the list that is shown in fig. 6. The numbers above the boxes are their indexes. We have a list containing the values a, b, c, d, e with an $n = 5$. For instance we perform an add: `add(2, z)`, the value `z` is inserted into our list at index 2 (as shown in fig. 7). The values after index 2, that is, $c, d,$ and e with indexes 2, 3, and 4 respectively, are *displaced*. Their new indexes are now $+1$ of their previous indexes, that is, they are now 3, 4, and 5 respectively.

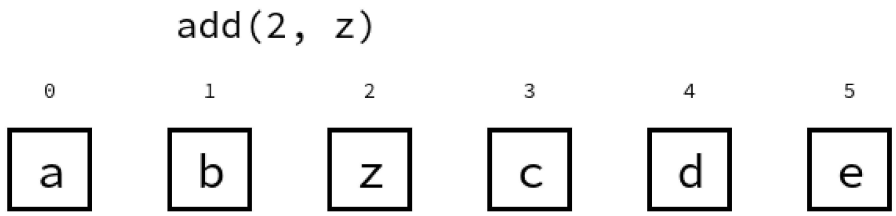


Figure 7: The resulting list after performing `add(2, z)`

Following fig. 7, if we were to perform `remove(1)`, the resulting list is shown in fig. 8. The value at index 1 is removed (b is removed) and the rest of the elements to the right of index 1 is displaced to take up the space of the removed element.

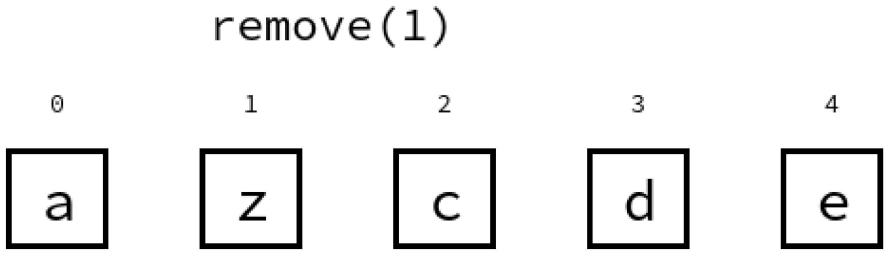


Figure 8: The resulting list after performing `remove(1)`

Notice that we can implement the Deque ADT with the following operations:

- `addFirst(x)` ⇒ `add(0, x)`
- `removeFirst()` ⇒ `remove(0)`
- `addLast(x)` ⇒ `add(size(), x)`
- `removeLast()` ⇒ `remove(size() - 1)`

The edge that Lists have over the Deque ADT is that we can access and modify any arbitrary element in the sequence. We aren't limited to accessing either ends of the sequence.

Unsorted Set, USet

The **USet** ADT represents an *unordered* and *unique* set of elements. This ADT mimics mathematical sets such that elements do not appear more than once. The USet supports the following operations:

- `size()` : returns the number of elements in the set, `n`
- `add(x)` : adds the element `x` to the set, provided that no element `y` exists in the set such that `y` $=$ `x`. Returns `true` if added into the set, else `false`.
- `remove(x)` : Removes `x` from the set, finds an element `y` such that `y` $=$ `x`. Returns `y`, else `null` if it doesn't exist.
- `find(x)` : Finds `x`, finds an element `y` such that `y` $=$ `x`. Returns `y`, else `null` if it doesn't exist.

Do note that 2 items can be *distinct* but are considered *equal*. This is important to note because it allows the creation of **dictionaries** or **maps**—a structure that *maps* **keys** to **values**.

To create dictionaries/maps, we create a compound object called a `Pair`, which is composed of a **key** and **value**. Two pairs are thus considered equal when their keys are equal. That is, a pair's value can be different and distinct but the pairs would be considered equal if their keys equal each other. For instance, if we store a `Pair(k, v)` in a USet, then call `find(x)` with `x` $=$ `Pair(k, null)` the result will be `Pair(k, v)`, since their keys (`k`) equal each other. In other words, this makes it possible to recover the value `v` given only the key `k`.

Sorted Set, SSet

The **SSet** ADT represents a sorted set of elements. An SSet stores elements in such a way that any 2 elements of the set can be compared. This ADT supports the same operations as a USet, with the important distinction of the `find(x)` operation:

- `find(x)` : finds `x` in the set, finds the smallest element `y` such that `y` $\geq$ `x`. Return `y`, else `null`.

This `find(x)` operation is sometimes called the **successor search**. It

comes with the benefit of returning a meaningful result even when no element equals `x` in the set.

The benefit of sorting the elements come at a cost, we'll see later on that USets offer better performance compared to SSets. When choosing which of these data structures to use, always use the USet, unless the additional feature of the SSet is really needed for your application.

Sitrep

The table below (tbl. 1) summarizes the ADTs and operations they support:

Table 1: Summary of ADTs and their operations

ADT	Operations
Queue	`add(x)`
	`remove()`
Stack	`push(x)`
	`pop()`
Deque	`addFirst(x)`
	`removeFirst()`
	`addLast(x)`
	`removeLast()`
List	`size()`
	`get(i)`
	`set(i, x)`
	`add(i, x)`
USet	`size()`
SSet	`add(x)`
	`remove(x)`
	`find(x)`

Array-based Lists

Finally we're getting to the meat of data structures—the implementation. We'll first take a look at implementing a data structure with a familiar concept—the array. In implementations that store data in an array, the array is called the **backing array**. Data structures that use backing arrays offer many advantages and at the same time are limited.

- Arrays offer constant (think $O(1)$) time access to any value in the array
- Arrays, by default, are *not* dynamic. Adding or removing an element from the middle would **displace** a *lot* of elements.
- Arrays cannot grow or shrink. When the number of elements in the list exceed the size of the backing array, a *new* array must be created and have data copied over *one-by-one*. This is actually quite an expensive operation.

Array Implementation

Basic C++ arrays don't keep track of their size/length. However, this would be quite convenient to have. Hence, we'll first create a straightforward wrapper implementation of an array that keeps track of its size/length.

```
#include<cassert>

template<class T>
class array {
    public:
        T* a;
        int length;
        ...
```

The size of the array is declared at the time of creation:

```
        ...
        array(int n) {
            length = n;
            a = new T[length];
        }
        ...
```

The elements of the array can be indexed, we overload the `[]` operator:

```
    ...
    T& operator[](int i) {
        assert(i >= 0 && i < length);
        return a[i];
    }
    ...
```

And when one array is assigned to another, we *consume* the source array. That is, the contents of the source array is *moved* into the destination array:

```
    ...
    array<T>& operator=(array<T> &b) {
        if (a != NULL) delete[] a;
        a = b.a;
        b.a = NULL;
        length = b.length;
        return *this;
    }
```

Our wrapper array implementation is complete. Let's move on to finally implementing some data structures.

ArrayStack

An `ArrayStack` implements the List ADT using an array `a`, also called the **backing array**. The list element with index `i` is simply stored in `a[i]`. Most of the time, the size of the backing array is larger than necessary. Therefore, an additional integer `n` is used to keep the actual number of elements stored in `a`. Thus list elements are stored in `a[0]`, `a[1]`, ..., `a[n - 1]`. This also means that `a.length` $\geq$ `n`.

```
template<class T>
class ArrayStack {
    array<T> a;
    int n;

    ...
```

```
int size() {
    return n;
}
```

`get(i)` and `set(i, x)`

Accessing and modifying elements inside the `ArrayStack` is a trivial task. We simply return or set `a[i]`, after checking bounds.

```
T get(int i) {
    assert(i >= 0 && i < n);
    return a[i];
}

T set(int i, T x) {
    assert(i >= 0 && i < n);
    T y = a[i];
    a[i] = x;
    return y;
}
```

`add(i, x)`

To implement the `add(i, x)` operation, let us visualize what it does to our backing array. Consider an `ArrayStack` with a backing array of length 10, containing the values `'a'` through `'g'`; hence, $n = 7$ (fig. 9).

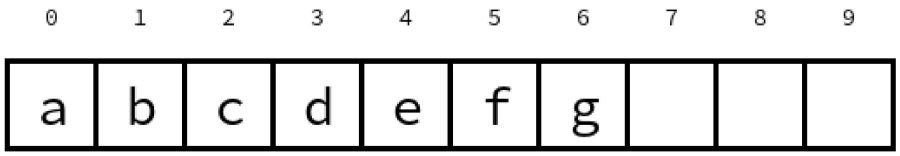


Figure 9: Initial visualization of our backing array

Upon performing an `add(2, 'h')` for instance, we first check whether our backing array `a` is full. If it is, we `resize()` the array and continue. How `resize()` is implemented will be discussed later. Since our `a` has enough space, we shift each of the elements to the right. We begin from the last index where we have a value stored until the index we want to add an element to. In this case, we shift the elements `a[n - 1]`, ..., `a[3]`, `a[2]` to the right (copy value of

`a[n - 1]` to `a[n]` and so on). A diagram of the shifting is shown in fig. 10.

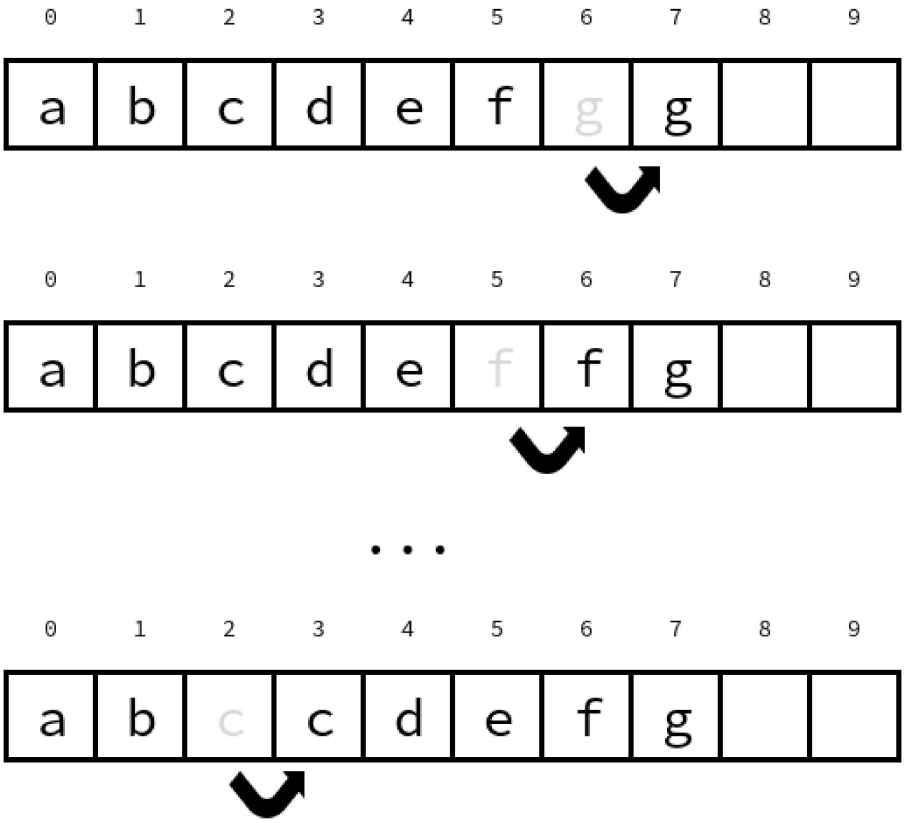


Figure 10: Shifting elements one-by-one in our backing array

Now that the index we want to add into (index 2) is free, we can finally set `a[2] = 'h'` and increment `n`. Done. The final state of our backing array after adding an element is shown in fig. 11.

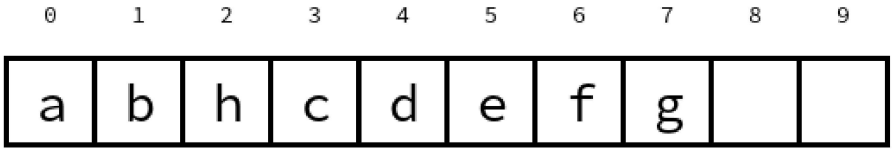


Figure 11: Our backing array after performing `add(2, 'h')`

```
void add(int i, T x) {
    assert(i >= 0 && i <= n);
    if (n + 1 > a.length)
        resize();
    for (int j = i; j > i; j--) {
        a[j] = a[j - 1];
    }
    a[i] = x;
    n++;
}
```

`remove(i)`

`remove(i)` is a similar operation to `add(i, x)`, such that we will have to shift the elements of our array to take the place of the removed element. For instance if we `remove(5)` from our previous example (fig. 11), we'll want to shift the elements `a[6]`, ..., `a[n - 1]` to the left (copy the value of `a[6]` to `a[5]` and so on).

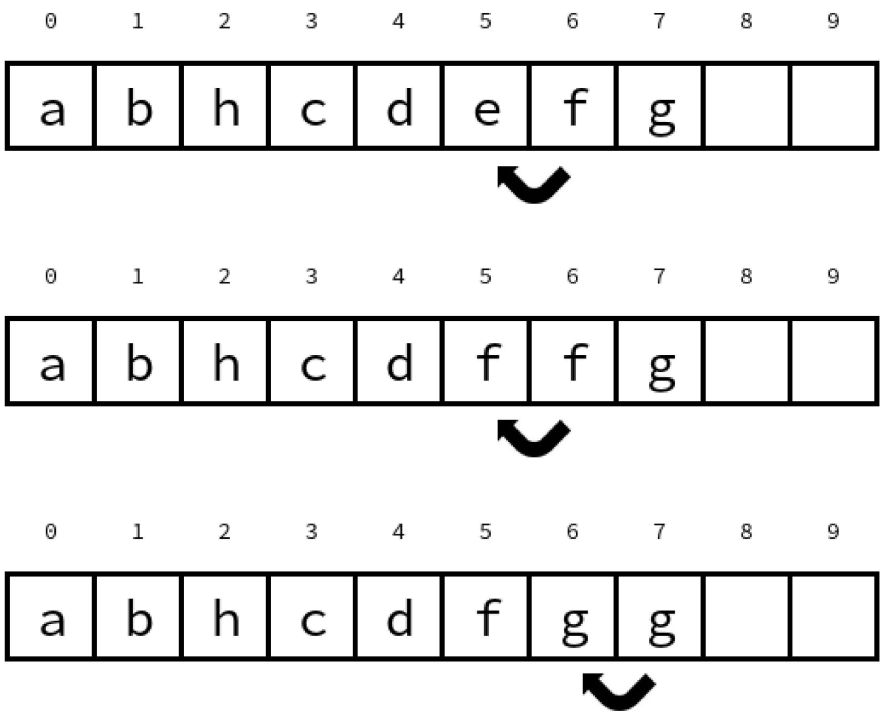


Figure 12: The changes the backing array goes through when performing `remove(5)`

Once we've shifted the elements to the left, we've by then removed the element at index `i` by overwriting it with the next element in the array. We also mustn't forget to reduce our `n` as well, to keep track of the number of elements in the list. Note that now our `n` doesn't represent

how many elements there *actually* are in the array. `a[7]` still contains the value `'g'` (see fig. 12), but this doesn't matter. Since as far as our `get(i)` and `set(i, x)` methods are aware of, the max index is `n` -1—effectively making `a[7]` inaccessible/deleted. Finally, we check whether `a.length` $\geq$ 3n, and if so, we'll need to resize the backing array `a` with the `resize()` function. This helps us reduce the amount of space we waste.

`resize()`

The `resize()` method creates a new array `b` whose length is $2n$ and copies the `n` elements from `a` into `b`. Then we set `a = b`, which sets our backing array to the new array.

```
void resize() {
    array<T> b(max(1, 2 * n));
    for (int i = 0; i < n; i++) {
        b[i] = a[i];
    }
    a = b;
}
```

Analysis

The `get(i)` and `set(i, x)` are both trivial and requires no loops to perform. Hence, we can say that both of these operations take constant time—$O(1)$.

On the other-hand, `add(i, x)` and `remove(i)` operations sometimes make a call to `resize()` which complicates things. For now, let us ignore calls to `resize()`. The cost of running `add(i, x)` and `remove(i)` is actually proportional to the number of elements we shift. We shift $n - i$ elements, hence $O(n - i)$.

Considering `resize()` calls, there exists a lemma:

If an empty `ArrayStack` *is created and any sequence of* $m \geq 1$ *calls to* `add(i, x)` *and* `remove(i)` *are performed, then the total time spent during all calls to* `resize()` *is* $O(m)$.

The proof can be found on p. 34 of the textbook.

Hence, calls to `resize()` are actually negligible and the *amortized* running time of `add(i, x)` and `remove(i)` is $O(1 + n - i)$ (constant time + shifting of the elements).

This makes the `ArrayStack` quite an efficient way to implement a Stack as well. As we can implement `push(x)` as `add(n, x)` and `pop()` as `remove(n - 1)`, resulting in an amortized running time of $O(1)$.

tbl. 2 summarizes the time complexity of the `ArrayStack`.

Table 2: Summarized running time of `ArrayStack` operations

Operation	Running time
`get(i)`	$O(1)$
`set(i, x)`	$O(1)$
`add(i, x)`	$O(1+n-i)$
`remove(i)`	$O(1+n-i)$

ArrayQueue

So we might notice that the `ArrayStack` is quite efficient when used as a Stack. This is because a Stack only has to work on one end of the list and elements at the end of our backing array take $O(1)$ constant time to add or remove. However, a FIFO Queue, on the other hand, must work with both ends of the list. Since the `ArrayStack` offers poor performance for working with the beginning of the backing array ($O(n)$), a FIFO Queue implemented with an `ArrayStack` will take at *least* $O(n)$ time to shift elements.

How then, do we implement an *efficient* array-based Queue? We first try to solve the problem of *always* shifting elements around, which would normally take $O(n)$ time. Let us consider that we have an infinite array `a`. Instead of shifting elements around, how about keeping track *which* index the array starts at? Instead of *always* starting from index 0, we keep track of an index `j`. Instead of storing values into the array at `a[0]`, `a[1]`, ..., `a[n - 1]` we now store them in `a[j]`, `a[j + 1]`, ..., `a[j + n - 1]`.

Initially, both `j` and `n` would be set to 0. To add an element to the queue, we set the value of `a[j + n]` and increment `n`. To remove an element from the queue, we simply increment `j` and decrement `n`.

This is all good and well, but the problem with this solution is the need for an infinite array. However, we can *simulate* the infinite array using a finite array and **modular arithmetic**.

If you aren't familiar with modular arithmetic, this is the kind of arithmetic when we talk about time of day. For example, 7:00 plus 10 hours is 5:00. We get 5:00 because $7 + 10 = 17 \equiv 5 \pmod{12}$ (also can be expressed as $17 \mod 12 = 5$). Less formally, 5 is the remainder we get when dividing 17 by 12.

Modular arithmetic is useful for simulating an infinite array because `i` mod `a.length` will always give a value in the range 0, ..., `a.length - 1`. Hence, we can treat our finite array more like a circular array by storing queue elements at array locations `a[j % a.length]`, `a[(j + 1) % a.length]`, ..., `a[(j + n - 1) % a.length]`. Now, even if we index beyond the size of our array, it will get *mapped* or *wrapped around* to the beginning of the array.

Of course, we have to make sure that the number of elements in the queue does not exceed the length of `a`. When the elements in the queue exceed `a`, we must `resize()` and reset `j` to start at 0 again.

```
template<class T>
class ArrayQueue {
    array<T> a;
    int j;
    int n;

    ...
```

A sequence of `add(x)` and `remove()` operations on an `ArrayQueue` is shown in fig. 13 to make things clear.

`add(x)`

To implement the `add(x)` operation we first check if `a` is full, if it is, we `resize()` the array, else we continue. Then we store `x` into `a[(j + n) % a.length]` and increment `n`.

`remove()`

To implement the `remove()` operation, we first keep the value of `a[j]` then increment `j` (modulo `a.length` by setting `j = (j + 1) % a.length`) and decrement `n`. If necessary, we `resize()` the array (when `a.length` ≤ 3 `n`). Then finally

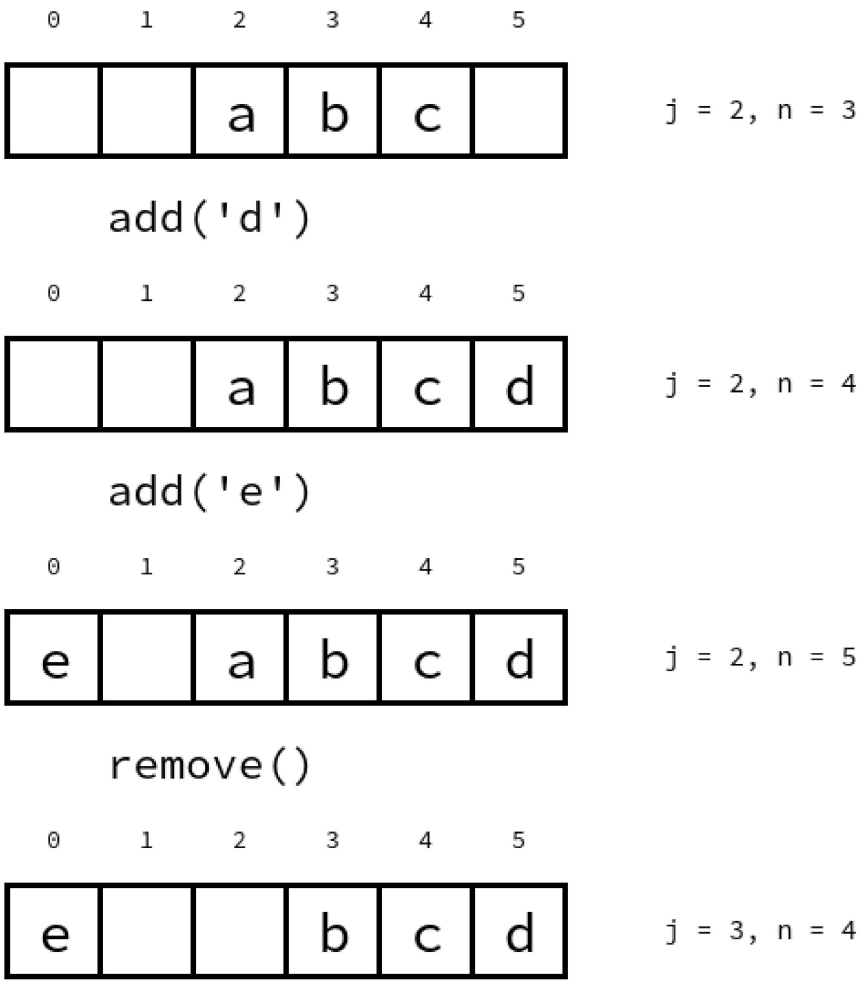


Figure 13: Sequence of `add(x)` and `remove()` on an `ArrayQueue`

return the stored value.

```
T remove() {
    T x = a[j];
    j = (j + 1) % a.length;
    n--;
    if (a.length <= 3 * n)
        resize();
    return x;
}
```

`resize()`

The `resize()` method is similar to the `ArrayStack`'s `resize()`. We allocate a new array `b` that is double our current `n` and we start copying from index `j % a.length`, `(j + 1) % a.length`, ..., `(j + n - 1) % a.length` onto `b[0]`, `b[1]`,..., `b[n - 1]` and set `j` to 0.

```
void resize() {
    array<T> b(max(1, 2 * n));
    for (int i = 0; i < n; i++) {
        b[i] = a[(j + i) % a.length];
    }
    j = 0;
    a = b;
}
```

Analysis

As you may notice, *all* of the operations of the `ArrayQueue` don't require looping through the array or anything. Ignoring calls to `resize()`, both of the operations `add(x)` and `remove()` take constant time—$O(1)$.

A summary of the time complexity of the `ArrayQueue` is found in tbl. 3.

Table 3: Summarized running time of `ArrayQueue` operations

Operation	Running time
`add(x)`	$O(1)$

Operation	Running time
`remove()`	$O(1)$

ArrayDeque

So we implement pretty good Stack and Queues, now we are implementing a List using an array. We'll be using the circular array technique we learned from the `ArrayQueue` so that we can get relatively good performance for `add(i, x)` and `remove(i)` for both ends. That is, we keep track of the beginning of the array with an index `j`

```
template<class T>
class ArrayDeque {
    array<T> a;
    int j;
    int n;
    ...
```

`get(i)` and `set(i, x)`

Both these operations are straightforward to implement. We offset our index by `j` and perform a modulo to *wrap around* to the beginning of the array when we reach the end.

```
T get(int i) {
    return a[(j + i) % a.length];
}

T set(int i, T x) {
    T y = a[(j + i) % a.length];
    a[(j + i) % a.length] = x;
    return y;
}
```

`add(i, x)`

To implement the `add(i, x)`, we first check if `a` is full and `resize()` if necessary. After that, we want our operation to shift as *few* elements as possible by shifting the least elements closer to an end of the array. We can do that by checking if `i` $<$ `n`$/2$. If

so, we shift the elements `a[j]`, ..., `a[(j + i) % a.length]` to the left by 1 position, updating `j` as well. Otherwise (if `i` $\geq$ `n`$/2$), we shift the elements `a[(j + i) % a.length]`, ..., `a[(j + n - 1) % a.length]` to the right by 1 position. By shifting elements this way, we guarantee that `add(i, x)` will never have to shift more than `min(i, n - i)` elements. In other words, we shift the half with *fewer* elements. We illustrate shifting the smaller half in fig. 14.

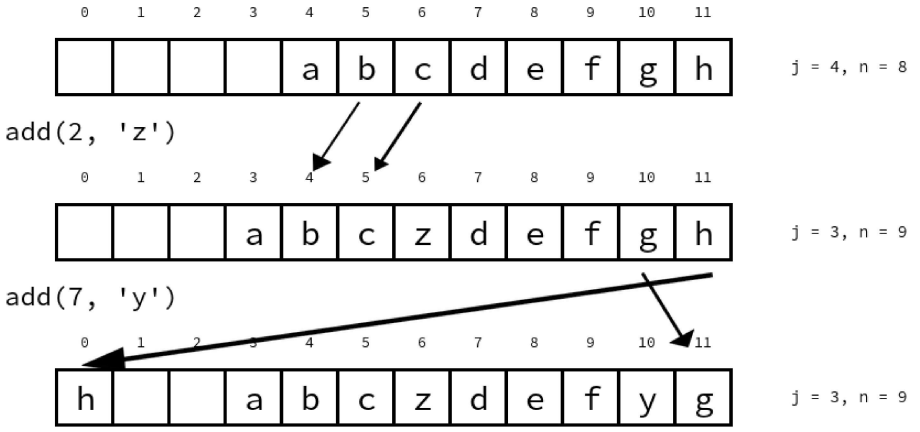


Figure 14: An `ArrayDeque` shifting the smaller half when adding elements in the middle

Additionally, if we were to `add(i, x)` at the actual ends of the array we wouldn't have to shift any element. For example, to implement `add(0, x)` we just decrement `j` (making sure to wrap `j` to `a.length - 1`), set `a[j]` to `x` and increment `n`. To implement `add(size(), x)`, we simply set `a[(j + n) % a.length]` to `x` and increment `n`.

```
void add(int i, T x) {
    if (n + 1 > a.length)
        resize();
    if (i == 0) {
        j = (j == 0) ? a.length - 1 : j - 1;
        a[j] = x;
        n++;
        return;
    }
    if (i == n) {
        a[(j + n) % a.length] = x;
        n++;
        return;
    }
    if (i < n / 2) {
        for (int k = j; k <= j + i; k++) {
            a[(k - 1) % a.length] = a[k %
 ↪  a.length];
        }
        j = (j == 0) ? a.length - 1 : j - 1;
    } else {
       for (int k = j + n - 1; k >= j + i; k--) {
            a[(k + 1) % a.length] = a[k %
 ↪  a.length];
        }
    }
    a[(j + i) % a.length] = x;
    n++;
}
```

`remove(i)`

The implementation of `remove(i)` is similar. We shift the lesser amount of elements to fill in the gap. If we perform a `remove(i)` with an `i` $<$ `n`/2, we shift `a[j]`, ..., `a[(j + i) % a.length]` to the right by one position, decrement `n`, and return the removed element. Otherwise (when `i` $>$ `n`/2), we shift `a[(j + i) % a.length]`, ..., `a[(j + n - 1) % a.length]` to the left by one position, decrement `n`, and return the removed element. The `remove(i)` operation never spends more than $O(1 + \min($`i`, `n` - `i`$))$ time to

shift elements.

Again, calling `remove(i)` at the actual ends of the array shouldn't require a move of any element at all. We can implement a `remove(0)` by simply incrementing `j`, decrementing `n`, and returning the removed element. Implementing `remove(size() - 1)` is also simple, we decrement `n` and return the removed element.

Analysis

Ignoring calls to `resize()`, which we has been proven to only add $O(m)$ amortized running time to m `add(i, x)` or `remove(i)` operations, both `add(i, x)` and `remove(i)` operations offer $O(1 + \min(\texttt{i}, \texttt{n} - \texttt{i}))$.

We summarize the running time of the operations offered by the `ArrayDeque` in tbl. 4.

Table 4: Summarized running time of `ArrayDeque` operations

Operation	Running time
`get(i)`	$O(1)$
`set(i, x)`	$O(1)$
`add(i, x)`	$O(1 + \min(\texttt{i}, \texttt{i} - \texttt{n}))$
`remove(i)`	$O(1 + \min(\texttt{i}, \texttt{i} - \texttt{n}))$